README 作成のための資料

Markdownの基本文法

よく使う書き方は次の10種類だけです。左の列のように書くと、GitHubでは右の列のように表示されます。

やりたいこと	書き方	表示のされ方・注意
見出し	`# 大見出し` `## 中見出し` `### 小見出し`	`#` の数が階層。`#` の後ろに半角スペースが必要。`#` はページに1つだけにする
段落	文章の間に**空行**を1行入れる	空行がないと、前の行とつながって1つの段落になる
段落内で改行	行末に半角スペース2つ、または ` `	GitHubでは、READMEの中で普通に改行しただけでは改行されない
太字・斜体・取り消し線	`**太字**` `*斜体*` `~~取り消し~~`	日本語の途中で太字が効かないときは、`**` の前後に半角スペースを入れる
箇条書き	`- 項目`（半角スペース2つで字下げすると子の項目）	`-` `*` `+` のどれでもよいが、1つにそろえる
番号付きリスト	`1. 手順` `2. 手順`	全部 `1.` と書いても自動で1, 2, 3と番号が付く
リンク	`[表示する文字](https://example.com)`	同じリポジトリのファイルは `[説明](Assets/Scripts/SavePoint.cs)` のように相対パスで書ける
画像	`![代わりの文字](docs/images/title.png)`	`!` を付けると画像になる。代わりの文字は、画像が表示されないときや読み上げに使われる
コード	文中は `` `GameState` ``、複数行は ` ```csharp ` と ` ``` ` で囲む	最初の ` ``` ` の後に `csharp` `bash` など言語名を書くと色が付く
表	`| 見出し | 見出し |` の次の行に `| --- | --- |`	`:---:` で中央寄せ、`---:` で右寄せ。セルの中で改行したいときは ` `
引用	`> 引用する文章`	左に縦線が入る。補足や注意書きにも使える
区切り線	`---`（前後に空行）	文章のすぐ下に空行なしで書くと、その文章が見出しになってしまう

GitHub独自の書き方

GitHubでは、基本文法に加えて次の書き方が使えます。どれも今回の README.md で実際に使っているので、見比べながら読んでください。

やりたいこと	書き方	使いどころ
チェックリスト	`- [ ] 未完了` `- [x] 完了`	「今後の改善予定」。完了したら `x` を入れると、継続して改善していることが伝わる
注意書き（Alerts）	`> [!NOTE]` の次の行から `> 本文`	種類は NOTE・TIP・IMPORTANT・WARNING・CAUTION の5つ。色とアイコンが付く。使いすぎは逆効果
折りたたみ	`<details>` `<summary>見出し</summary>` 本文 `</details>`	長い補足や、読まなくてもよい手順をしまう。`<summary>` の直後に空行を入れないと中のMarkdownが効かない
画像の大きさ指定	`<img src="docs/images/title.png" width="360">`	Markdownの `![]()` では大きさを指定できないのでHTMLで書く
中央寄せ	`<p align="center">…</p>`	タイトルやメイン画像に使う。`style="…"` のCSSはGitHubでは無視される
バッジ	`![Unity](https://img.shields.io/badge/Unity-6000.3-000000?logo=unity)`	shields.io で作る小さなラベル画像。使った技術が一目でわかる。多くても5個程度
ページ内リンク（目次）	`[遊び方](#遊び方)`	見出しの文字がそのままリンク先になる。英字は小文字、スペースは `-`、記号（`・` `/` など）は消える
図（Mermaid）	` ```mermaid ` で始めて `flowchart LR` と矢印 `A --> B`	画面の流れや処理の流れを、画像を作らずに図にできる
コメント（非表示）	`<!-- メモ -->`	自分用のメモ。ファイルには残るが画面には出ない
絵文字	`:rocket:` または 🚀 を直接入力	タイトルに1つ程度なら華やかになる。見出しごとに付けると読みにくい

ポートフォリオ向けREADMEの構成

「見た目 → 何ができるか → どう作ったか → 何を学んだか」の順に並べるのが基本です。採用担当者は上から読んで途中でやめるので、大事なものほど上に置きます。

順	項目	書くこと	コツ
1	タイトル・一言説明・バッジ	作品名と、1文で「どんなゲームか」	「○○する△△ゲーム」の形にすると伝わりやすい
2	メイン画像（GIF）	実際に動いている様子	**最重要。**文章を読まなくても完成度が伝わる
3	概要	制作の背景、期間、人数、**自分の担当範囲**	共同制作なら担当を必ず書く。面接で必ず聞かれる
4	遊び方・操作方法	ルールとキー操作	操作は表にすると一目でわかる
5	主な機能	実装した機能の一覧	「攻撃できる」ではなく「長押しでチャージし、攻撃力が2倍」のように具体的に
6	技術的なポイント	自分で考えた仕組みを図やコードで説明	**一番差がつくところ。**「なぜそう作ったか」まで書く
7	開発環境	Unity・言語・パッケージのバージョン	バージョンまで正確に書く
8	実行方法	クローンから起動までの手順	他の人がその通りにやって動くか、一度確かめる
9	苦労した点と解決方法	問題・原因・解決の3点セット	未経験者はここで**問題解決の力**を見られる
10	今後の改善予定	やりたいことをチェックリストで	自分のコードを客観的に見られていることの証明になる
11	学んだこと	身に付けた技術や考え方	技術名だけでなく「何ができるようになったか」を書く
12	使用素材・クレジット	画像・音の作者と配布元	書かないと「他人の素材を自分のものとして見せている」と受け取られる